

蔡禹煊答辩闭环：AST 与公共数据结构

1. 先记住自己的定位

你这部分的核心不是“把语义分析和代码生成讲完”，而是守住两件事：

1. AST 作为统一中间表示，为什么有必要。
2. AST 节点和公共数据结构怎么组织，为什么后续阶段都依赖它。

最短自我介绍可以直接说：

我主要参与的是 AST 与公共数据结构这一层。输入是语法分析阶段归约出的语法结构，输出是统一的 AST 节点对象和数组、record、参数这些公共类型信息。这样语义分析和代码生成就都可以围绕同一棵树工作，而不需要各自重新解释语法结构。

2. 现场先打开哪些文件

建议只记这 2 个文件，够用了：

1. [src/ast.h](#)
2. [src/ast.cpp](#)

打开顺序建议：

1. 先看 `ASTNode / ExpressionNode / StatementNode / DeclarationNode`，讲“节点分层”。
2. 再看 `ProgramNode` 和 `CompoundStatementNode`，讲“整棵程序树怎么组织”。
3. 最后看 `ASTVisitor` 和 `accept()`，讲“为什么 semantic 和 codegen 能共用同一套节点”。

3. 老师最可能问什么

问题 A：AST 是什么？为什么一定要有 AST？

先打开：

- [src/ast.h:80](#)

回答模板：

AST 是抽象语法树，是整个编译流水线共享的中间表示。

parser 负责把 token 按文法归约成这棵树，后面的语义分析和代码生成都不再直接面对 token，而是统一遍历 AST。

这样做的好处是语法结构和后续处理解耦，后面要加 `record`、`repeat-until`、错误定位时，不需要每个阶段都重新理解一遍语法。

最重要的一句话：

parser 只建树，semantic 检查树，codegen 映射树。

问题 B: AST 节点怎么分层?

先打开:

- [src/ast.h:84](#)
- [src/ast.h:96](#)
- [src/ast.h:251](#)
- [src/ast.h:410](#)

回答模板:

整个 AST 的根基类是 `ASTNode`, 里面放了所有节点共有的位置信息 `line` 和 `column`。
在它上面又分成三大类: `ExpressionNode` 表达式、`StatementNode` 语句、`DeclarationNode` 声明。
这样后面无论是语义分析还是代码生成, 都能按语法类别处理, 而不是所有节点混在一起。

如果老师追问“这样分有什么好处”:

因为表达式、语句、声明的处理方式本来就不同。比如表达式有类型和值类别, 语句强调执行结构, 声明强调名字和类型信息。分层之后后续访问逻辑更自然。

问题 C: 程序整体在 AST 里怎么表示?

先打开:

- [src/ast.h:453](#)
- [src/ast.h:272](#)

回答模板:

整个程序对应 `ProgramNode`。它里面有三块主要内容: 程序名、声明列表 `declarations`, 以及主程序体 `main_body`。
主程序体是一个 `CompoundStatementNode`, 里面再顺序保存语句列表。
所以从 AST 角度看, 整个 Pascal-S 程序就是“声明区 + 主体语句区”的树形结构。

这段最好顺手点一下:

- `ProgramNode`
- `declarations`
- `main_body`
- `CompoundStatementNode::statements`

问题 D: 你们 AST 怎么体现数组、record、函数调用这些扩展结构?

先打开:

- [src/ast.h:157](#)
- [src/ast.h:174](#)
- [src/ast.h:229](#)
- [src/ast.h:36](#)

- [src/ast.h:57](#)

回答模板:

AST 里数组访问用 `ArrayAccessNode` 表示, 里面保存数组名和下标表达式列表。

`record` 字段访问用 `RecordAccessNode` 表示, 左边是一个表达式, 右边是字段名, 所以天然支持多级字段访问。

函数调用用 `FunctionCallNode` 表示, 带参数列表。

同时, 数组和 `record` 相关的公共信息不只在 AST 节点里, 还在 `ArrayInfo`、`RecordField`、`RecordInfo` 这些公共数据结构里保存, 供语义分析和代码生成复用。

问题 E: 为什么要用访问者模式?

先打开:

- [src/ast.h:471](#)
- [src/ast.cpp:5](#)

回答模板:

`ASTVisitor` 的作用是把“节点结构”和“处理逻辑”拆开。

节点本身只负责保存结构, 并通过 `accept()` 把自己交给访问者; 真正怎么检查、怎么生成代码, 不写在节点类里, 而是分别交给 `SemanticAnalyzer` 和 `CodeGenerator`。

这样同一棵 AST 可以被多个阶段复用, 不会把所有逻辑写死在节点类里。

如果老师追问“`accept` 在做什么”:

`accept()` 本质上就是双分派入口。比如 `AssignmentNode::accept()` 会调用 `visitor.visit(*this)`, 从而把当前具体节点类型派发给对应阶段的处理函数。

4. 最该会指给老师看的 5 段代码

4.1 `ASTNode` 和位置信息

位置:

- [src/ast.h:84](#)

怎么讲:

每个节点都继承自 `ASTNode`, 里面统一保存 `line` 和 `column`。这保证了后面的语义错误和语法错误都能回到源码中的具体位置。

4.2 `ProgramNode`

位置:

- [src/ast.h:453](#)

怎么讲:

`ProgramNode` 是整棵树的根节点。它负责把程序名、声明列表和主程序体组织在一起，是后续 semantic 和 codegen 的统一入口。

4.3 CompoundStatementNode

位置：

- [src/ast.h:272](#)

怎么讲：

复合语句也就是 `begin ... end`，在 AST 里被整理成一个语句列表节点。后面无论是语义分析还是代码生成，都可以顺序遍历这个 `statements` 数组。

4.4 RecordAccessNode

位置：

- [src/ast.h:174](#)

怎么讲：

`record` 字段访问不是简单字符串拼接，而是一个真正的 AST 节点。左边是 `record_expr`，右边是 `field_name`，所以像 `point.inner.y` 这种多级字段访问会自然形成嵌套节点结构。

这段对你们“record 扩展”很关键。

4.5 ASTVisitor + accept()

位置：

- [src/ast.h:474](#)
- [src/ast.cpp:5](#)

怎么讲：

`ASTVisitor` 定义了所有节点的访问接口，`ast.cpp` 里的 `accept()` 则把具体节点派发出去。这样语义分析和代码生成都能复用同一棵树，但它们的逻辑彼此独立。

5. 如果老师一个点一个点追问，怎么答

5.1 AST 和 parse tree 有什么区别？

回答：

我们这里用的是抽象语法树，不是完整保留所有文法细节的解析树。AST 只保留后续阶段真正需要的语义结构，比如表达式、语句、声明和它们之间的关系，这样更适合作为编译流水线的中间表示。

5.2 AST 输出的是什么呢？

回答：

输出不是文本，也不是直接执行结果，而是一组内存中的节点对象。语义分析和代码生成都在这些节点对象上工作。

5.3 你们的 AST 怎么和 parser 连接？

回答：

在 Bison 的归约动作里，直接构造这些 AST 节点。比如识别到赋值语句就构造 `AssignmentNode`，识别到函数调用就构造 `FunctionCallNode`，最后把整棵树挂到 `ProgramNode` 上。

这里他如果要翻代码，可以让周柄名翻 `parser.y`，你这边不用硬背动作细节。

5.4 公共数据结构除了 AST 节点还有什么？

回答：

除了节点类本身，我们还定义了 `ArrayDimension`、`ArrayInfo`、`RecordField`、`RecordInfo`、`ParameterInfo` 这些公共结构。它们一方面在 AST 节点里保存必要信息，另一方面也会被语义分析和代码生成复用。

6. 老师如果让你“讲设计思路”，你就这么收

我们这一层的设计思路是，把 Pascal-S 程序先抽象成统一的 AST 和公共类型结构，让 parser、semantic、codegen 三个阶段围绕同一套数据工作。这样可以减少阶段之间的耦合，也能更自然地支持数组、record、函数调用和新语句扩展。

7. 最短保底版：30 秒回答

如果他非常紧张，只背这段也够过关：

我主要参与的是 AST 与公共数据结构这一层。AST 是整个编译器共享的中间表示，parser 负责构造它，semantic 和 codegen 都遍历它。我们把节点分成表达式、语句、声明三大类，同时定义了数组、record、参数这些公共结构，用来支撑后续类型检查和代码生成。相关代码主要在 `ast.h` 和 `ast.cpp`。

8. 最容易说错的 4 个点

1. 不要把 AST 说成“最终输出”。

正确说法：AST 是中间表示。

2. 不要把访问者模式讲成“只是设计模式作业”。

正确说法：它解决的是 semantic 和 codegen 共用同一棵 AST 的问题。

3. 不要把 `record` 结构翻译放到 AST 里讲太深。

正确说法：AST 负责表示 `record` 字段访问结构，真正映射成 `struct` 是 codegen 做的。

4. 不要把 parser、semantic、codegen 的职责混掉。

正确说法：parser 建树，semantic 检查树，codegen 生成树对应的目标代码。

9. 蔡禹煊只需要记住的文件定位

最推荐只记下面这些：

- [src/ast.h:80](#)
- [src/ast.h:84](#)
- [src/ast.h:174](#)
- [src/ast.h:272](#)
- [src/ast.h:453](#)
- [src/ast.h:474](#)
- [src/ast.cpp:5](#)

这 7 处足够覆盖他的大部分答辩问题。